

论符号计算系统

——表征、抽象代数语义与 E-Graph 工程框架

(本项目研究性文档，作者可自行补充)

2026 年 01 月

论符号计算系统

——表征、抽象代数语义与 E-Graph 工程框架

(本项目研究性文档，作者可自行补充)

2026 年 01 月

本文档为研究/探索型项目资料，随项目演进持续更新。
如需引用，请注明项目目录：symbolic_computation_engine。

献给：喜欢把“形式系统”当作工程系统来实现的人。

序

本文档是对一个独立探索项目 (`symbolic_computation_engine`) 的稳定论述。研究重点不是某一类具体任务 (例如函数求导), 而是符号计算系统本身:

- 用怎样的表达式表征承载推导与化简;
- 如何把抽象代数 (环、模、理想、商结构) 作为语义边界与等价关系来源;
- 如何在工程上选用合适的数据结构与计算框架, 使系统可扩展、可调试、可演进。

由于处于早期研究阶段, 本书将问题域刻意收敛到:

- 多项式表达式 (变量、常数、加减乘、幂的可选扩展);
- 圆括号作为顺序与嵌套结构 (用以表达组合与优先级, 而不是作为独立算子);
- 与之对应的代数语义: 环 (**ring**)、模 (**module**)、理想 (**ideal**) 与商环/商模 (**quotient**)。

本书对 E-Graph 的引用与借鉴, 仅限于其表征方式与等价维护方法 (E-Class/E-Node、Union-Find、hash-cons、extraction/提取), 并以此作为系统设计的核心坐标。

Contents

序	vii
1 研究范围与系统契约	1
1.1 输入语言：多项式与括号嵌套	1
1.2 系统输出：化简、规范化与等价证明的工程接口	1
2 表征问题：AST、DAG 与 E-Graph 的分工	2
2.1 AST：语法直观，但等价维护与共享是外部工作	2
2.2 DAG：共享子表达式的工程表征	2
2.3 E-Graph：把等价关系变成数据结构	3
3 E-Graph 表征细节：E-Class、E-Node 与 Extraction	4
3.1 E-Node 的数据形态：操作符 + 子等价类	4
3.2 E-Class 的意义：等价类作为“语义单元”	4
3.3 Extraction：从等价集合中选一个代表式	4
4 抽象代数语义：环、理想与商结构如何进入系统	6
4.1 为什么要显式引入抽象代数？	6
4.2 环上的多项式：基础代数语义	6
4.3 理想与商环：以“约束”定义新的等价	7
4.4 多项式代数操作的理论基础与工程实现	7
4.4.1 表达式树结构与 DAG/E-Graph 表征	7
4.4.2 结合律、交换律、分配律	7
4.4.3 同类项合并	7
4.4.4 多项式标准化	7
4.4.5 多项式展开与因式分解	8
4.4.6 代数重写规则	8
4.4.7 多项式加减乘除算法	8
4.4.8 符号化简与规范化	8
4.4.9 小结	8

5	工程实现框架：面向数据的 E-Graph 原型（与本项目代码对照）	9
5.1	为什么采用 ECS/SoA?	9
5.2	轻量 E-Graph 的最小构成	9
6	结语：可持续的研究路线	10
A	附录：与本项目文件的对应关系（索引）	11
	参考文献	12

Chapter 1

研究范围与系统契约

1.1 输入语言：多项式与括号嵌套

我们研究的表达式族可以理解为一个极简语法（示意）：

$$E ::= E + E \mid E - E \mid E \times E \mid (E) \mid x \mid c$$

其中 x 为变量， c 为常数（本阶段可取整数或有理数；工程原型中可暂用浮点，但语义上建议以精确数为目标）。括号“(E)”并不是一个算子，它只表示抽象语法树/图的嵌套结构。

1.2 系统输出：化简、规范化与等价证明的工程接口

一个早期的符号系统至少需要能回答三类问题：

- 构造：如何把输入表达式转成内部 IR；
- 化简/规范化：如何得到更短、更结构化的表达；
- 等价：如何表达并维护“A 与 B 等价”的证据（至少是可追踪的规则应用链，或可重放的重写过程）。

本书把“等价维护”视为第一等公民，这也是选择 E-Graph 表征的原因。

Chapter 2

表征问题：AST、DAG 与 E-Graph 的分工

2.1 AST：语法直观，但等价维护与共享是外部工作

AST（树）在解析阶段具有天然优势：结构与括号直接对应。但一旦进入化简阶段：

- 共享子表达式需要额外做 CSE（公共子表达式消除）；
- 等价关系往往通过“顺序敏感”的 rewrite 管线维护；
- 任何一次替换都可能破坏之前的结构共识，难以调试与复用。

因此，AST 是良好的入口表征，但不是理想的推理工作台。

2.2 DAG：共享子表达式的工程表征

DAG（有向无环图）将表达式中的“引用”变成可共享结构。最常见的工程实现是一种结构驻留（**hash-cons**）：

- 为每个节点构造一个可哈希的 key: `(op, child_1, child_2, ...)`；
- 相同 key 复用同一个节点；
- 从构造之初就得到共享结构。

对于多项式类表达式，这个技巧尤其重要：例如 $(x + y)^2$ 若不共享，展开与合并时会引入大量重复子结构。

注 2.1. DAG 解决的是结构共享问题，但它并不主动维护“形式不同但语义等价”的关系。要回答 $x + x$ 与 $2x$ 的等价，需要额外的推理层。

2.3 E-Graph: 把等价关系变成数据结构

E-Graph 的核心贡献在于，它把“等价”内化为结构：

- **E-Class**: 一个等价类，代表“同一个意义”；
- **E-Node**: 一个顶层构造（例如加法、乘法），其孩子指向 E-Class；
- **Union-Find**: 维护 E-Class 的合并；
- **hash-cons**: 去重 E-Node（基于孩子 e-class 的 key）。

这使得：当我们说“某处引用了一个 E-Class”，实际上是在引用一组等价表达式的集合。

Chapter 3

E-Graph 表征细节：E-Class、E-Node 与 Extraction

3.1 E-Node 的数据形态：操作符 + 子等价类

定义 3.1 (E-Node). 一个 *E-Node* 表示表达式的顶层形状：

$$enode = (op; c_1, c_2, \dots, c_k)$$

其中 op 是操作符（如 $+$ 、 $\times$ ）， c_i 是 *E-Class* 的标识。

在多项式语义下，通常需要额外标记：

- 操作符的结合性与交换性（如加法/乘法通常是交换且结合）；
- 常量与变量节点的载荷（例如整数系数、变量名）；
- 项的有序/无序表示（例如加法项列表是否需要排序）。

3.2 E-Class 的意义：等价类作为“语义单元”

E-Class 可以理解为“某个语义值的名字”。它的工程意义是：

- 任何重写规则的目标不是“替换树”，而是合并等价类；
- 多条规则并行加入结构，不需要决定严格的执行顺序；
- 最终通过提取（extraction）选择一个人类/机器更偏好的代表形式。

3.3 Extraction：从等价集合中选一个代表式

定义 3.2 (Extraction). 给定一个 *E-Graph* 与一个代价函数 $\text{cost}(\cdot)$ ，提取过程输出某个表达式 e ，使得：

$$e \in [E] \quad \text{并且} \quad \text{cost}(e) \text{ 最小}$$

其中 $[E]$ 表示目标 *E-Class* 所代表的等价表达式集合。

对于早期系统，代价函数可以极简：节点数、乘法数、最大嵌套深度等。但从研究角度看，代价函数是把抽象代数语义与工程目标连接起来的接口。

Chapter 4

抽象代数语义：环、理想与商结构如何进入系统

4.1 为什么要显式引入抽象代数？

多项式化简不是“字符串替换”，它依赖清晰的语义边界：

- 系数来自哪个环（如 $\mathbb{Z}$ 、 $\mathbb{Q}$ 、有限域 $\mathbb{F}_p$ ）；
- 变量集合是什么；
- 允许的等价关系有哪些（例如是否引入某个理想的生成元）。

抽象代数给出的不是“更多规则”，而是更结构化的规则来源。

4.2 环上的多项式：基础代数语义

我们可以把表达式解释为环 R 上的多项式环 $R[x_1, \dots, x_n]$ 的项。在这种语义下，基本等价来自环公理：

- 加法交换/结合；乘法交换（对交换环）/结合；分配律；
- 0 与 1 的单位性质；
- 加法逆元（负号）。

在工程上，这意味着：

- 允许将 $x + x$ 合并为 $2x$ ；
- 允许将 $(a + b) + c$ 重排为 $a + (b + c)$ ，并进一步“扁平化”为项列表；
- 允许对项排序以得到某种规范形（例如按字典序排列单项式）。

4.3 理想与商环：以“约束”定义新的等价

引入理想 $I \triangleleft R[x]$ 后，商环 $R[x]/I$ 将把“差落在理想里”的两个多项式视为等价。例如，给定生成元集合 $G = \{g_1, \dots, g_m\}$ ，若 $f - g \in \langle G \rangle$ ，则在商环语义下 $f \equiv g$ 。

注 4.1. 这给 *E-Graph* 提供了关键接口：等价关系不止来自“基础公理”，还来自“用户引入的约束/生成元”。工程上，这意味着规则库是可配置的，而不是写死的。

4.4 多项式代数操作的理论基础与工程实现

本节系统梳理多项式代数运算的主要操作，分别从抽象代数理论与 DAG/E-Graph 工程实现角度进行说明。

4.4.1 表达式树结构与 DAG/E-Graph 表征

理论基础：多项式表达式可用树（AST）或有向无环图（DAG）结构表示，叶节点为变量或常数，内部节点为加、减、乘、除、幂等运算符。抽象代数中，这对应于多项式环 $R[x_1, \dots, x_n]$ 的项结构。

工程实现：DAG 通过结构驻留（hash-cons）实现子表达式共享，E-Graph 则进一步将等价类（E-Class）作为节点引用，实现表达式的等价关系维护。

4.4.2 结合律、交换律、分配律

理论基础：加法、乘法满足结合律与交换律，乘法对加法满足分配律：

$$\begin{aligned}(a + b) + c &= a + (b + c) \\ a + b &= b + a \\ a(b + c) &= ab + ac\end{aligned}$$

这些律源自环的公理，是多项式化简与重写的基础。

工程实现：在 E-Graph 中，这些律通过重写规则（rewrite rules）实现。例如，`add(a, b) -> add(b, a)`、`mul(a, add(b, c)) -> add(mul(a, b), mul(a, c))` 等。规则的应用会合并等价类，实现表达式的规范化。

4.4.3 同类项合并

理论基础：同类项指变量幂次、名称完全相同的项。合并同类项即将系数相加，如 $2x^2 + 3x^2 = 5x^2$ 。

工程实现：在 DAG/E-Graph 中，通过模式匹配和重写规则（如 `add(k1 * a, k2 * a) -> mul(add(k1, k2), a)`）自动完成同类项合并。

4.4.4 多项式标准化

理论基础：多项式标准化包括项排序（如按幂次降序）、变量顺序固定等，便于比较和进一步化简。

工程实现：E-Graph 的 extraction 阶段可结合代价函数（如节点数、嵌套深度、排序规则）选取规范化表达式作为代表。

4.4.5 多项式展开与因式分解

理论基础：展开是将乘积形式转为和的形式（如 $(x+1)(x-2) = x^2 - x - 2$ ），因式分解则反之。理论上依赖分配律、提取公因式等。

工程实现：展开通过分配律相关重写规则实现。因式分解可通过逆向规则或专门的因式分解算法（如提取公因式、配方法等）实现。

4.4.6 代数重写规则

理论基础：代数重写规则如 $a^0 = 1$ 、 $a^1 = a$ 、 $0 \cdot a = 0$ 、 $1 \cdot a = a$ 等，源自环的单位元、零元等性质。

工程实现：这些规则以 DSL 或代码形式写入规则库，E-Graph 在 rewrite pass 阶段批量应用，自动完成表达式的简化。

4.4.7 多项式加减乘除算法

理论基础：加减法通过同类项合并，乘法通过分配律展开，除法可用多项式长除法或带余除法。

工程实现：加减乘通过重写规则和 DAG 结构实现。除法可通过专门的算法实现，也可在 E-Graph 中通过规则（如 $\text{div}(\text{mul}(a, b), b) \rightarrow a$ ）进行部分约简。

4.4.8 符号化简与规范化

理论基础：通过不断应用上述规则，将多项式化为最简、标准或指定形式。

工程实现：E-Graph 支持等价饱和（equality saturation），即反复应用所有可用规则，最终通过 extraction 选取最优代表式，实现符号化简与规范化。

4.4.9 小结

上述各类操作，理论上均有抽象代数的公理或推论支撑，工程上则依赖 DAG/E-Graph 结构与规则库的协同，实现自动化、可扩展的多项式符号计算。

Chapter 5

工程实现框架：面向数据的 E-Graph 原型（与本项目代码对照）

5.1 为什么采用 ECS/SoA?

研究型系统的工程约束往往是：

- 需要大量扫描与批处理（例如匹配模式、应用规则、重建索引）；
- 需要可追踪与可调试（每一步都能解释为什么合并）；
- 需要可扩展（增加新算子/新理论时，不希望重写整个对象体系）。

因此，本项目倾向“面向数据”的表格化存储（SoA/ECS 风格）：节点 ID 是数组下标，每列数组是一种属性。

5.2 轻量 E-Graph 的最小构成

本项目现有原型（`symbolic_engine.py`）可以被视作一个轻量 E-Graph 思路的工程落点：

1. **NodeTable**：用 NumPy 数组列存储节点（`op`、`a`、`b`、`value`、`var_id`）；
2. **hash-cons**：构造期 DAG 共享；
3. **Union-Find**：维护等价类（E-Class）；
4. **enode index**：以 $(op, e(a), e(b))$ key 做同余合并；
5. **rewrite pass**：一组基础规则（目前示例为 0/1 与常量折叠），未来可替换为多项式/理想语义规则；
6. **extraction**：按代价函数从等价类提取代表式。

注 5.1. 为了契合本书研究范围，后续规则库应逐步替换为“多项式 + 理想/商结构”的化简规则，而不扩展到求导等任务。

Chapter 6

结语：可持续的研究路线

这本小册子尝试建立一个稳定坐标系：

- 先用多项式与括号嵌套把系统核心机制跑通；
- 用抽象代数（环/理想/商结构）定义等价关系的边界；
- 用 E-Graph（Union-Find + hash-cons + extraction）作为等价维护框架；
- 工程上用面向数据（ECS/SoA）实现可扩展、可批处理的推理平台。

在此基础上，未来的扩展不应以“堆更多规则”为主要方式，而应以更清晰的理论分层 + 更可控的数据结构与代价模型推进。

Appendix A

附录：与本项目文件的对应关系（索引）

- 原型实现: `symbolic_engine.py`
- 公共入口: `symbolic.py`
- 演示脚本: `demo.py`
- 回归测试: `test_symbolic_engine.py`
- 聊天记录: 相关资源/符号计算:AST 与 DAG 表达式...-20260107183224.txt
- 转载资料（仅借鉴 E-Graph 表征部分）: 相关资源/转载: 为什么函数求导之后表达式长度大大超出原来的表达式 - 酱紫君的回答.md

参考文献

Bibliography

- [1] Willsey, M. *et al.* “egg: Fast and Extensible Equality Saturation.” (2019).
<https://arxiv.org/abs/1904.02990>
- [2] 酱紫君（转载笔记，仅借鉴 E-Graph 表征部分）. “为什么函数求导之后表达式长度大大超出原来的表达式？”
<https://www.zhihu.com/question/609058716/answer/1954152484693058426>